czii-cryo-et-object-identification

5th Place Solution

Rank	5
Team	Youssef Ouertani
Author	Youssef Ouertani
Topic ID	561580
Version	Original

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561580
Retrieved with Kaggle CLI on 2026-10-21.

Introduction

First and foremost, I would like to express my gratitude to Allah, the competition hosts, Kaggle, and @hengck23 for making this event possible. This competition provided an exciting opportunity to work with 3D volumetric data and develop an efficient solution for particle detection.

I present my straightforward approach to solving the problem. Below, I outline the key steps of my solution, including data preparation, network architecture, training strategy, and inference techniques. I also discuss what worked and what didn't, along with the final results achieved on the public and private leaderboards.

Data Preparation & Loading

Volume Normalization

- The volumes were normalized by calculating the (5, 99) percentiles of the 7 volume datasets and averaging them to perform min-max scaling.

Label Preparation

- The labels were created as spheres with a radius of log2(given_radius) * 0.8.

Training Data

- The model was trained on batches of 4 patches, each of size 128x128x128.
- Patches were randomly sampled from the volumes during training.

Data Augmentation

- Flipping along all 3 axes.
- Rotations of 90°, 180°, and 270° along the z-axis.
- Mean and standard deviation shifting using the following function

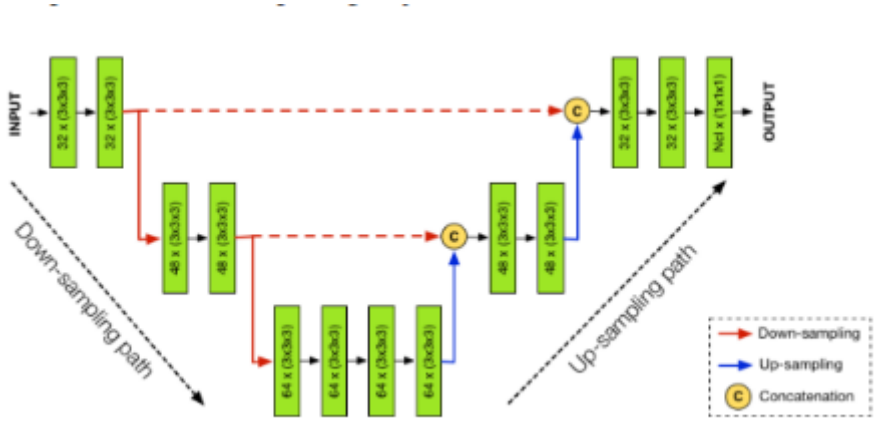
```
():
   factor =  / (shift * )
   std = image.std()
   mean = image.mean()
   shift_mean = (torch.rand() / factor - shift).item()
   shift_std = (torch.rand() / factor - shift).item()
   new_mean = mean + mean * shift_mean
   new_std = std + std * shift_std
   new_image = (image - mean) / std * new_std + new_mean
    new_image
```

Network Architecture

The network architecture is inspired by DeepFinder, with the following modifications:

- Added a BatchNorm3d layer as the first input layer.
- Reduced the number of channels to 28, 32, and 36, resulting in a compact model size of 1.44 MB.
- Used trilinear interpolation for downsampling and upsampling, except for the final upsampling layer, which uses a transposed convolution.

Here is a visualization of the architecture:



Training Strategy

- Optimizer: Adam with a learning rate of 0.0001, beta1 of 0.9, and beta2 of 0.999.
- Loss Function: Label smoothing cross-entropy with a smoothing factor of 0.01.
- Precision: Training was conducted in float16 precision with gradient clipping applied.
- Model Ensembling: The final model consists of 4 seeds of the above architecture, trained on all 7 volumes.

Inference

- Patch Splitting:

For inference, the volumes were split into patches of size 128x128x128 with minimal overlap along the z-axis and overlap + 1 along the x and y axes.

- Test-Time Augmentation (TTA):

Applied 3 flips and 3 rotations.

- Post-Processing:

Connected components were applied to binary masks generated using a probability threshold for each particle.
Components with an area less than 1/7th of the trained masks were removed.

Results

- Public Leaderboard: 0.7798
- Private Leaderboard: 0.7825

What Didn’t Work

- Multicascade Network: This approach did not yield improvements.
- Larger Models: These models tended to overfit quickly and performed worse than the compact architecture.